SENIOR PROJECT:
MATCHMAKERS PROJECT DOCUMENTATION

Kathryn Jerez, Front End Developer

Jacob Nickerson, Back End Developer

George Stewart, Network Developer

Professor Ashish Aggarwal, Advisor, ashishjuit@ufl.edu

University of Florida

https://youtu.be/OKC9UYMJlnc

April 21, 2026

ABSTRACT

Modern financial markets rely on High-Frequency Trading (HFT), yet the field remains inaccessible to students due to extreme financial risk and the high cost of proprietary software. We present an educational HFT system designed for risk-free experimentation. The platform features a high-performance Rust matching engine capable of processing hundreds of thousands of orders per second, paired with a configurable data simulation server. By integrating industry-standard protocols like FIX and ITCH with a user-friendly Python API, the system provides a realistic environment for students to develop, backtest, and visualize algorithmic strategies under various market conditions.

KEYWORDS

High Frequency Trading, Asynchronous I/O, Low-Latency, Financial Information eXchange (FIX) 4.2,

MoldUDP64, NASDAQ TotalView-ITCH 5.0,

TABLE OF CONTENTS

I. PROJECT OVERVIEW....4
A. Project Goals and Objectives....4
B. Team Members....4
II. DELIVERABLES....5
A. Completed Deliverables....5
B. Functional Specifications....6
C. Current Project Status....7
III. TECHNICAL DETAILS....8
A. Codebase Overview....8
Repository Structure....8
Version Control & Branches....8
B. Key Dependencies and Libraries....9
C. Deployment Instructions....9
D. Software Architecture....10
IV. TESTING INFORMATION....10
A. Testing Conduction....10
B. Test Cases and Results....11
C. Outstanding Bugs and Defect Log....12
V. USER DOCUMENTATION....13

A. User Guide 13
B. FAQ 22
Simulator 22
Limit Order Book 23
User Interface 24
FIX (Financial Information eXchange) 25
MoldUDP64 26
ITCH 26
VI. TRANSITION PLAN 27
A. Knowledge Transfer 27
B. Next Steps 27
VII. ACKNOWLEDGEMENTS 28
VIII. APPENDICES 29
XI. BIOGRAPHY 29

I. PROJECT OVERVIEW

This project addresses the significant barriers blocking students from engaging with high-frequency trading. By creating an extensible platform with a Rust-based matching engine and a Python API, we offer a practical way for students to develop a deeper understanding of trading dynamics.

A. Project Goals and Objectives

- Build a High-Performance Matching Engine: Build a core engine using Rust to ensure low-latency performance.
- Create a User-Friendly Python API: Enable students to write and test trading algorithms using Python.
- Provide a Risk-Free Simulation Environment: Deliver a controlled sandbox where users can observe and test complex trading dynamics without financial risk.

B. Team Members

Name	**Role**	**Responsibilities**
Jacob Nickerson	Back End Developer	Core HFT engine (matching, order books), performance optimization, market data replay/simulation server, trading strategies.
George Stewart	Network Developer	Network protocol development and client-server communication.
Kathryn Jerez	Front End Developer	Design and develop the GUI for visualizing market data, testing and experimenting with different trading strategies, submitting orders, and viewing results.

II. DELIVERABLES

A. Completed Deliverables

- Simulator (Rust):
  - Limit Order Book: Orders are entered into a full depth limit order book w/ per-order granularity. Trades are executed automatically by price crossing in price-time priority.
  - Simulated latency effects: Configurable simulated latency allows for a constant time offset for latency and a randomly sampled time offset for jitter
  - Random order generation via poisson distribution: Orders are randomly generated with inter-arrival times randomly generated from an exponential distribution. Other order parameters are randomly sampled, such as type from a uniform distribution and price from a normal distribution
  - File-based order replay: A flow of orders can be recorded and logged to a binary file. This file can be replayed as a source of data.
  - Simulated time with real-time mode: Simulator uses a discrete-time model that is decoupled from wall-clock time. Real-time mode allows Simulator to pace events to wall-clock speed
  - Parameter configuration via command-line: All features above are configurable with different values and modes via command-line arguments.
- Networking Protocol Implementations (Rust):
  - FIX 4.2: TCP-based session management logic (logon, heartbeats, sequence management, etc.) and order routing messages (New Order, Cancel, Update, etc.).
  - NASDAQ TotalView-ITCH 5.0: Binary encoding and decoding for market event updates (Add, Execute, Delete, etc.)

  - MoldUDP64: UDP-based transport layer and sequence management (retransmission requests) for market data broadcasting.
- User Interface (Python):
  - Market Data Visualization: Processes MoldUDP64 market events to populate real-time candlestick charts and price updates.
  - Order Book Visualization: Displays order book (showing bid/ask depth and spread changes) directly from HFT engine.
  - Order Management: Order submissions (both from bots and users) and cancellations are sent as FIX messages to the server for execution.
  - Strategy Execution: Supports creating and running trading bots that execute custom strategies either developed or imported into the application.
  - Performance Analytics Visualization: Displays live strategy metrics and statistics.

B. Functional Specifications

- Price-Time Priority:
  - Orders are handled in order of price primarily and in order of time within the same price level. Time is recorded in nanoseconds for maximum granularity. In the event of a tie, the simulator discretely handles one event at a time and assigns a unique identifier on receipt, which is used to break double ties.
- Deterministic Matching:
  - Using data replay or seeded runs allows deterministic flows of orders. Since orders are matched strictly on price matches and ordered via price-time priority, the same flow of orders will always result in the same execution of trades.

- Real-Time Visualization:
    - When the application connects to the server, a processing loop begins that continuously drains the MoldUDP64 event feed. Each incoming event is processed by the matching engine to update the order book state, while also being passed to the candlestick chart controller.
- Bot Management:
    - When the application is running, each bot is managed by its own runner that keeps track of its state, orders, and performance. During the processing loop, market events are sent to each active bot, which allows them to place or update orders through the methods implemented in their respective strategy.
- Strategies:
    - Strategies are written as Python classes that define how a bot reacts to the market data. Once a bot is active and the strategy code is validated, the runner creates an instance of the strategy and attaches the bot object. That way, the strategy is able to access certain helper methods, important for actions such as submitting orders. Each method is called by the runner in its part of the lifecycle (on_start when the bot begins running, on_book when there is a change in the book state, on_trade if the incoming event is a trade, etc.)

C. Current Project Status

The project has the majority of its stated features from the original plan. The engine supports a single symbol high-cap equity and the application supports user strategies and performance visualization. However, some alterations and modifications were made to some. Of note: historical replay mode exists by proxy in file replay mode, but true L3 data is difficult to access. A tool was planned for conversion of L2 to L3 by interpolating a valid order flow between data points, but it is still a work in progress.

III. TECHNICAL DETAILS

A. Codebase Overview

The HFT system is consolidated into one unified GitHub repository containing both the Rust and Python components of the project. The Rust components are organized into three crates within the crate directory, while the Python application is located in the gui directory.

Repository Structure

- crates/mm_core: This crate is the unified library for all three Rust crates. It contains shared data structures and logic used by the server and client. It contains the fix_core, itch_core, lob_core, and moldudp64_core modules.
- crates/engine: This crate is the Exchange Server engine. It contains the logic for the simulator, matching engine, server-side networking implementations of FIX, ITCH, and MoldUDP64, and implementations for different sources of synthetic data.
- crates/pyclient: This crate is the Trading Client. It contains the logic for the client-side limit order book and the client-side networking implementations of FIX, ITCH, and MoldUDP64. It also contains the PyO3 bindings to bridge the Rust-based client to the Python user interface.
- gui: This is the Python-based User Interface component.

Version Control & Branches

The project is managed using Git for version control, hosted on GitHub. Our main branch is the stable, production-ready codebase. All merges into main require a Pull Request and a dedicated branch for any changes requested. Feature Branches (feat/) are branches for developing new features for the codebase. Fix Branches (fix/) are branches for resolving any bugs found in the main branch.

B. Key Dependencies and Libraries

Category	**Dependencies**	**Functional Purpose**
Interoperability	pyo3, pyo3-stub-gen	Python bindings and type-stub generation for high performance Rust FFIs in Python.
User Interface	lightweight-charts, PyQt5, pandas, numpy, pywebview, pyqtgraph, QScintilla, PyQtWebEngine	GUI toolkits and libraries for high-performance, cross-platform, and customizable application components.
Stochastic Modeling	rand_distr, fastrand	Distribution sampling and fast pseudorandom number generation for market simulation.
Asynchronous I/O	mio, socket2	Event-driven network polling and low-level socket configuration.
Binary De/Serialization	rykv, serde	High performance serialization of struct packed data to files
Memory Management	bytes, ringbuf, circular-buffer	Lock-free SPSC buffers and fixed-capacity caches for O(1) data access.

C. Deployment Instructions

The project is hosted on GitHub and can be cloned like a typical repository using `git clone git@github.com:jacobnickerson/money-matcher.git`. The `main` branch is used for releases, whereas other branches are only used for development. For simply running the project, stick to the main branch. Compilation is handled by a build script, but some additional dependencies must be installed prior to building. These are: `rustc`, `python`, and `Qt5`. Installation of these depends on your operating system, but using Fedora Linux as an example:

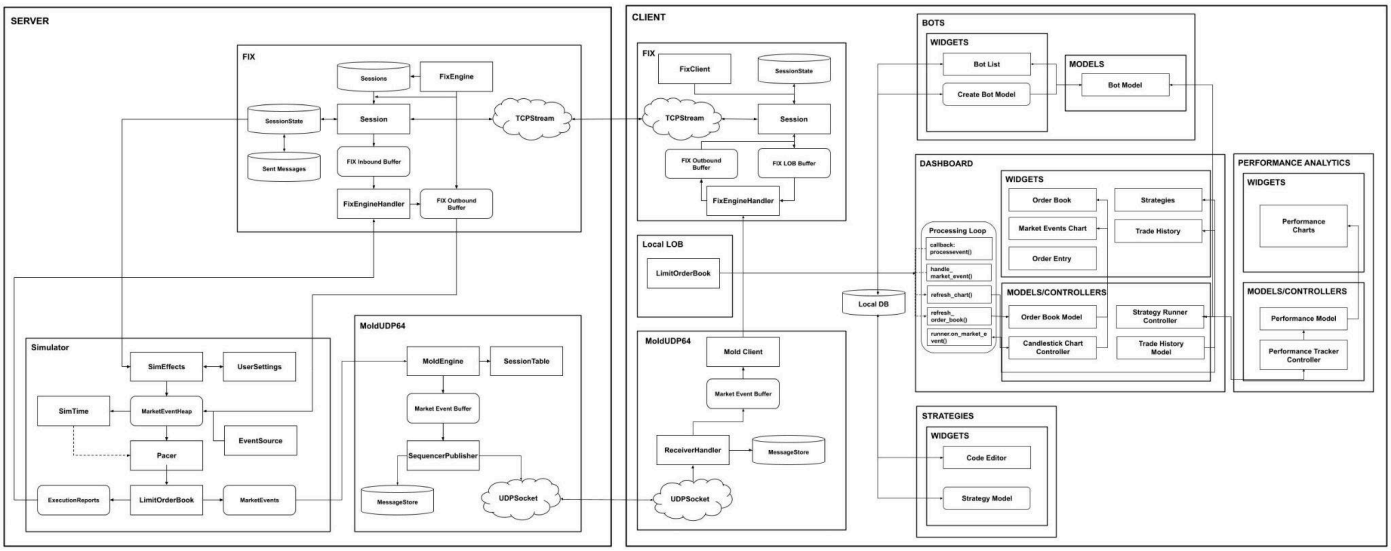
```sh
sudo dnf install cargo python qt5-devel
```

A release version of the project can be compiled using `make release`, which will install python requirements and compile the engine and the python module.

The application can be run using `python3 src/gui/main.py`.

The exchange server is run as a separate process. The exchange server can be run using `./target/release/engine`. Configuration of the server is done primarily via command-line arguments. More documentation on these options is included below.

D. Software Architecture



IV. TESTING INFORMATION

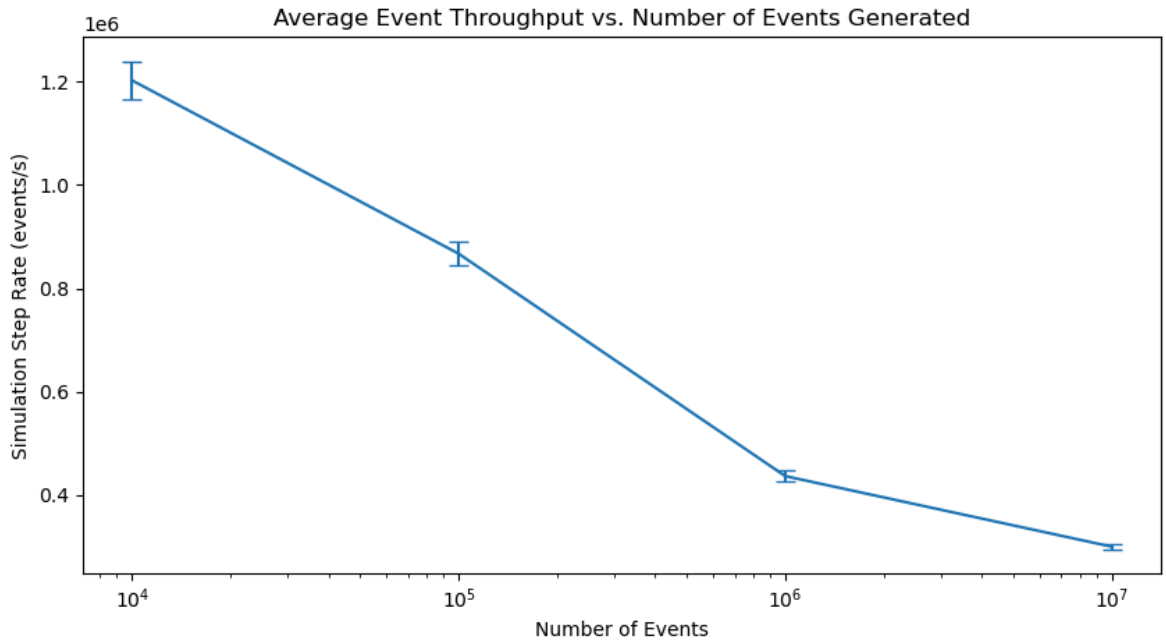
A. Testing Conduction

The system uses automated unit testing to ensure deterministic behavior, reliability, and protocol compliance across the core exchange and networking layers. For the networking layer, module-level unit tests validate byte-level encoding and decoding of FIX, ITCH, and MoldUDP64 messages. These tests also verify system and communication reliability by confirming accurate sequence number tracking and session

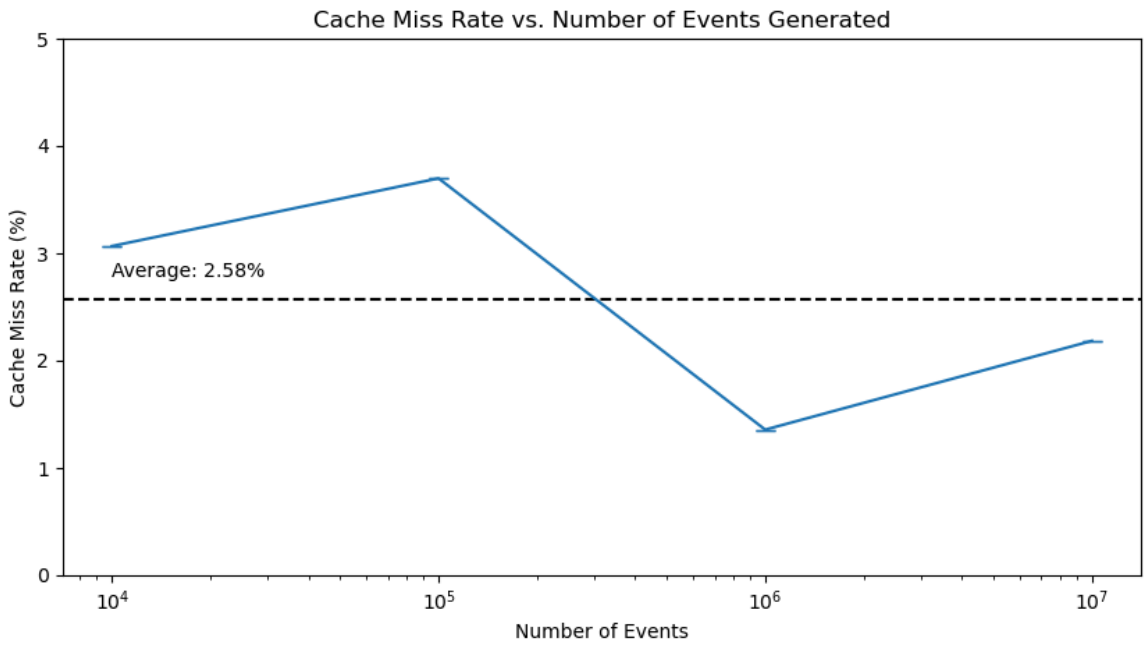
recovery following dropped packets. Within the matching engine, tests validate that orders correctly update the limit order book, maintain price-time priority, and trigger the removal of price levels as orders are matched. A Continuous Integration pipeline is implemented through GitHub Actions, which executes this test suite on every pull request, blocking merges that lead to the failure of any unit test.

B. Test Cases and Results

Performance tests were done to benchmark the throughput and performance of the simulator. The testing was performed on local hardware (Ryzen 7800x3d), however strict isolation of processes was not performed. The tests were run on NixOS, with the process niceness set to -19. For each event count, the test was run 50 times. The seed `67` was used for all test runs. The runtime was measured strictly from the time of the first event generation to the last event generation and does not include things like setup. Notably, as the limit order book gets saturated with orders, the throughput decreases.



The cache miss-rate was also benchmarked against the number of events. Cache misses and cache references were measured using `perf stat`.



C. Outstanding Bugs and Defect Log

There is an underflow error related to the python client limit order book. The suspected issue is due to the separation of market events and trade events. When a market event is made that matches against another, in between the receipt of that event and the receipt of the trade event, the best ask and best bid can cross resulting in an unsigned underflow when calculating spread.

V. USER DOCUMENTATION

A. User Guide

Engine

The engine comes with many command-line arguments for configuration, which are shown above. These options can be used to configure simulated effects, RNG, and constrain the runtime to real-time. Additionally, `--record` can be passed to record all events into a binary serialized file. This is used for the file replay mode, which will be discussed later. A screenshot of the help menu for the engine is shown below:

```
Usage: engine [OPTIONS] <COMMAND>

Commands:
  poisson  Generate synthetic orders by randomly sampling inter-arrival times from an exponential distribution
  file     Replay a historical record of order data from a file, file must contain binary data logged using --record
  help     Print this message or the help of the given subcommand(s)

Options:
      --sim-latency <SIM_LATENCY>
          Simulated latency in nanoseconds

          [default: 0]

      --sim-jitter-type <SIM_JITTER_TYPE>
          Simulated jitter sampling type

          [default: none]
          [possible values: none, uniform, normal]

      --low <LOW>
          Simulated jitter uniform distribution lower bound

      --high <HIGH>
          Simulated jitter uniform distribution upper bound

      --mean <MEAN>
          Simulated jitter normal distribution mean

      --std-dev <STD_DEV>
          Simulated jitter normal distribution standard deviation
```

```
      --seed <SEED>
          RNG seed for randomly sampled values, if unspecified a random one is picked

      --real-time
          Attempt to run the simulation in real-time by attempting to keep sim time and real time synchronized

      --logging
          Log errors and initialization steps to stdout

      --record
          Log events to a binary file for use with file-based replay

      --benchmark
          Records runtime and events processed and outputs to stdout after simulator finishes generating orders

          Output is in CSV format: step_count,run_time(nanosec),sim_time(nanosec)

  -h, --help
          Print help (see a summary with '-h')

  -V, --version
          Print version
```

There are currently two commands, which correspond to different data generation types, each of which has their own options. Exactly one command must be given to the engine.

The first option is poisson. Poisson randomly generates orders by sampling random distributions for different parameters. Primarily, inter-arrival time is sampled using an exponential distribution. The options allow for the configuration of all the different sampled values, including the rate of orders, the proportion of bids vs asks, the average prices for each type, and the proportion of orders for each type (limit, market, cancel, update). A screenshot is attached below showing the help menu for poisson mode.

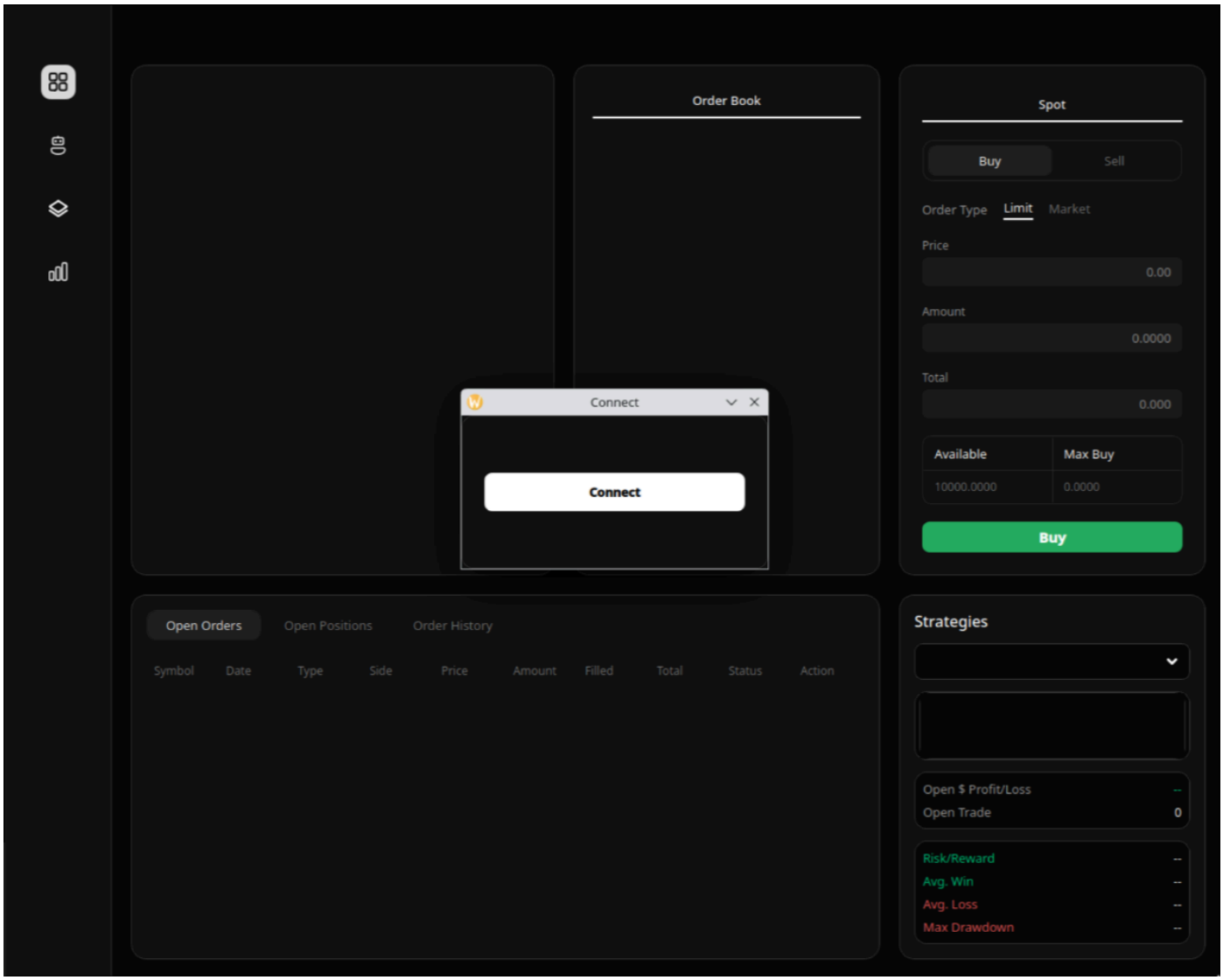
```
Generate synthetic orders by randomly sampling inter-arrival times from an exponential distribution

Usage: engine poisson [OPTIONS]

Options:
      --count <COUNT>
          Number of orders to generate before terminating, if unused the simulation runs indefinitely
      --order-rate <ORDER_RATE>
          Rate of production of synthetic orders in orders per second [default: 100000]
      --bid-rate <BID_RATE>
          Proportion of synthetic orders that are bids vs asks, value must be between 0-1 [default: 0.5]
      --new-limit-rate <NEW_LIMIT_RATE>
          Proportion of synthetic orders that are new limits, value must be between 0-1 and must sum to 1 with the other event types [default: 0.5]
      --cancel-rate <CANCEL_RATE>
          Proportion of synthetic orders that are cancels, value must be between 0-1 and must sum to 1 with the other event types [default: 0.4]
      --market-rate <MARKET_RATE>
          Proportion of synthetic orders that are market orders, value must be between 0-1 and must sum to 1 with the other event types [default: 0.05]
      --update-rate <UPDATE_RATE>
          Proportion of synthetic orders that are updates, value must be between 0-1 and must sum to 1 with the other event types [default: 0.05]
      --bid-avg-price <BID_AVG_PRICE>
          Average order price in cents for bids, must be a positive, non-zero value [default: 1000]
      --bid-price-dev <BID_PRICE_DEV>
          Standard deviation of order price in cents for bids, must be a positive, non-zero value [default: 50]
      --ask-avg-price <ASK_AVG_PRICE>
          Average order price in cents for asks, must be a positive, non-zero value [default: 1000]
      --ask-price-dev <ASK_PRICE_DEV>
          Standard deviation of order price in cents for asks, must be a positive, non-zero value [default: 50]
  -h, --help
          Print help
```

The other mode is file mode. File mode allows replaying of data from a binary-encoded file. These files can be generated by running the engine with the `--record` flag. Additionally, the batch size for file-reads can be tuned for performance. The help menu for file mode is shown below:

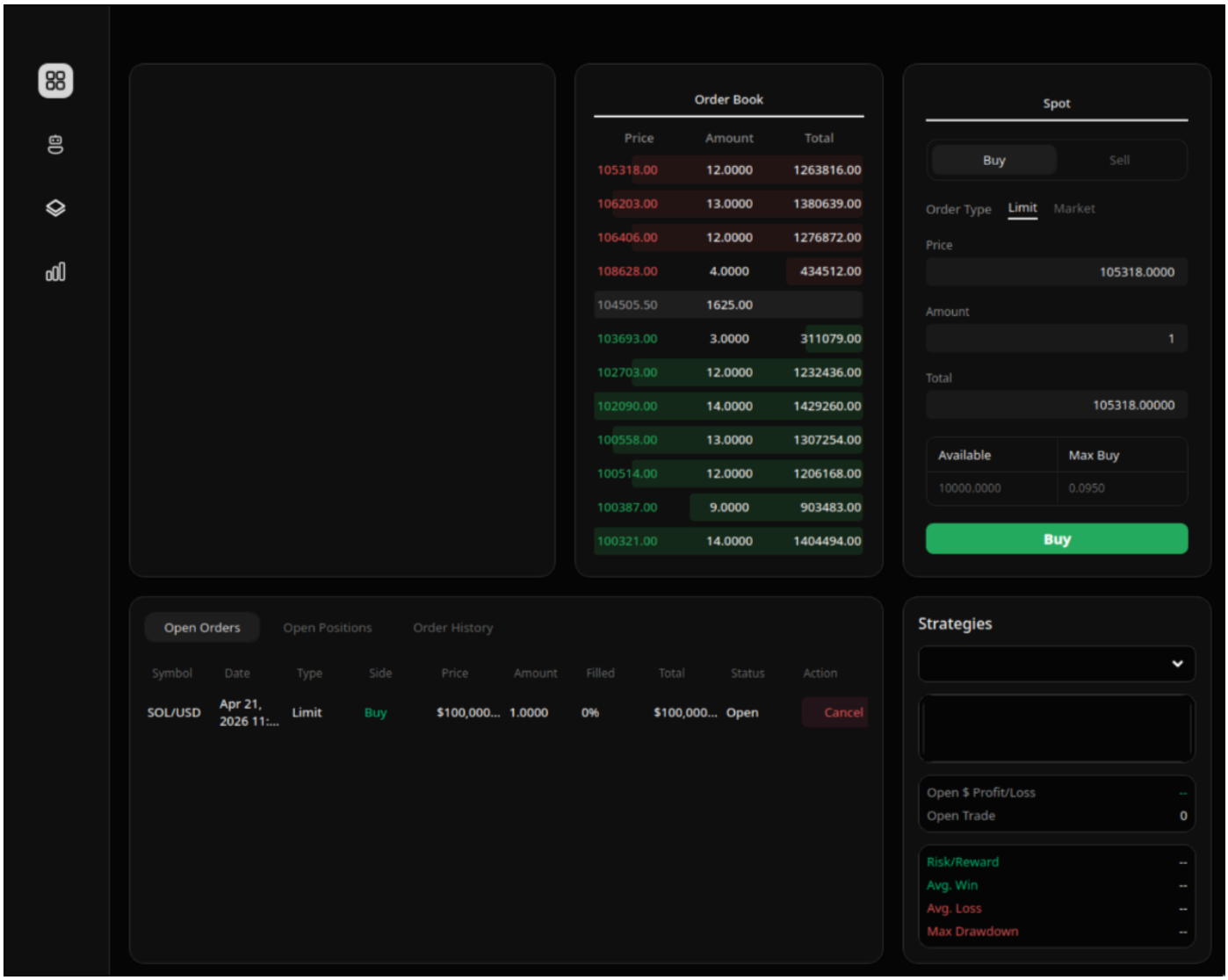
```
Replay a historical record of order data from a file, file must contain binary data logged using --record

Usage: engine file [OPTIONS] --file-name <FILE_NAME>

Options:
      --file-name <FILE_NAME>    File path to file containing binary-mapped order data
      --batch-size <BATCH_SIZE>  Batch size for batch-reading from file [default: 64]
  -h, --help                     Print help
```

Client

On launch, the client will show the following connect prompt:

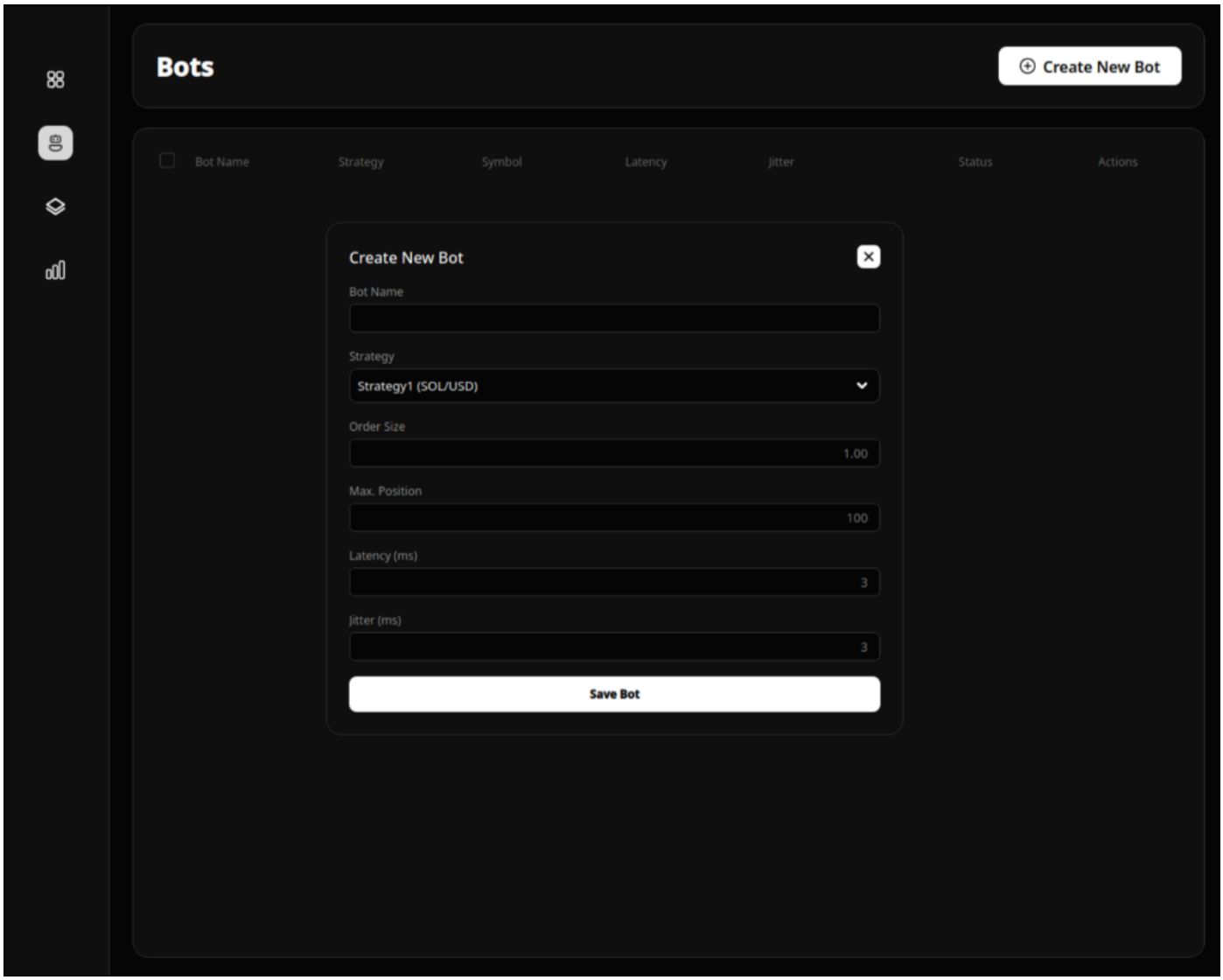


Connect sends a logon message to the engine and starts a session. It should be noted that some synchronization issues can occur when the client joins late, so for minimal friction, start the client first, start the engine, then connect. Below is a screenshot when the server is running:

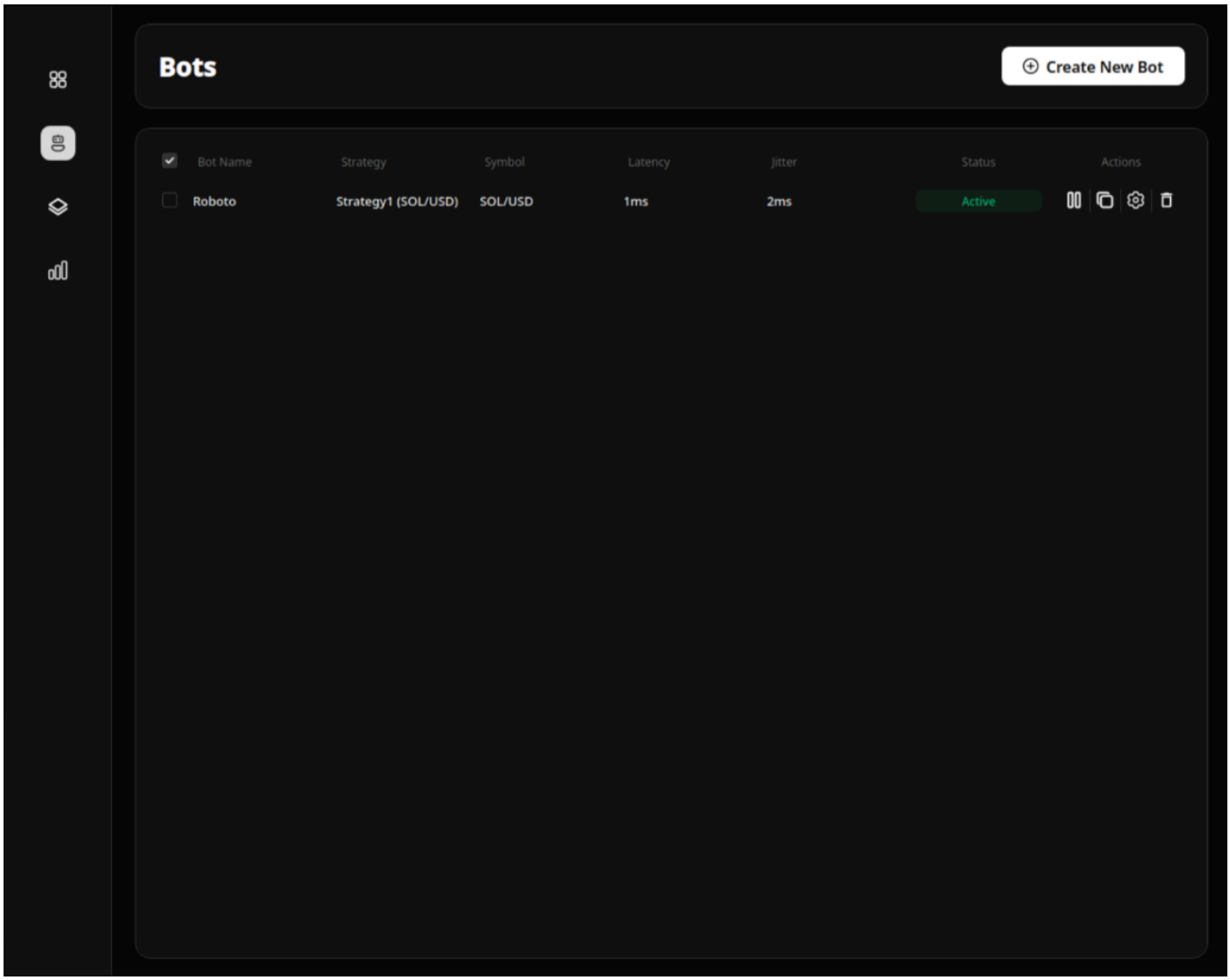


The Order Book window shows prices and liquidity. Red prices denote asks, while green denotes bids and gray denotes the mid price. The spot window allows manual entry of orders. At the bottom there is a window to view currently open orders, as well as history. The strategy window can be used to monitor and

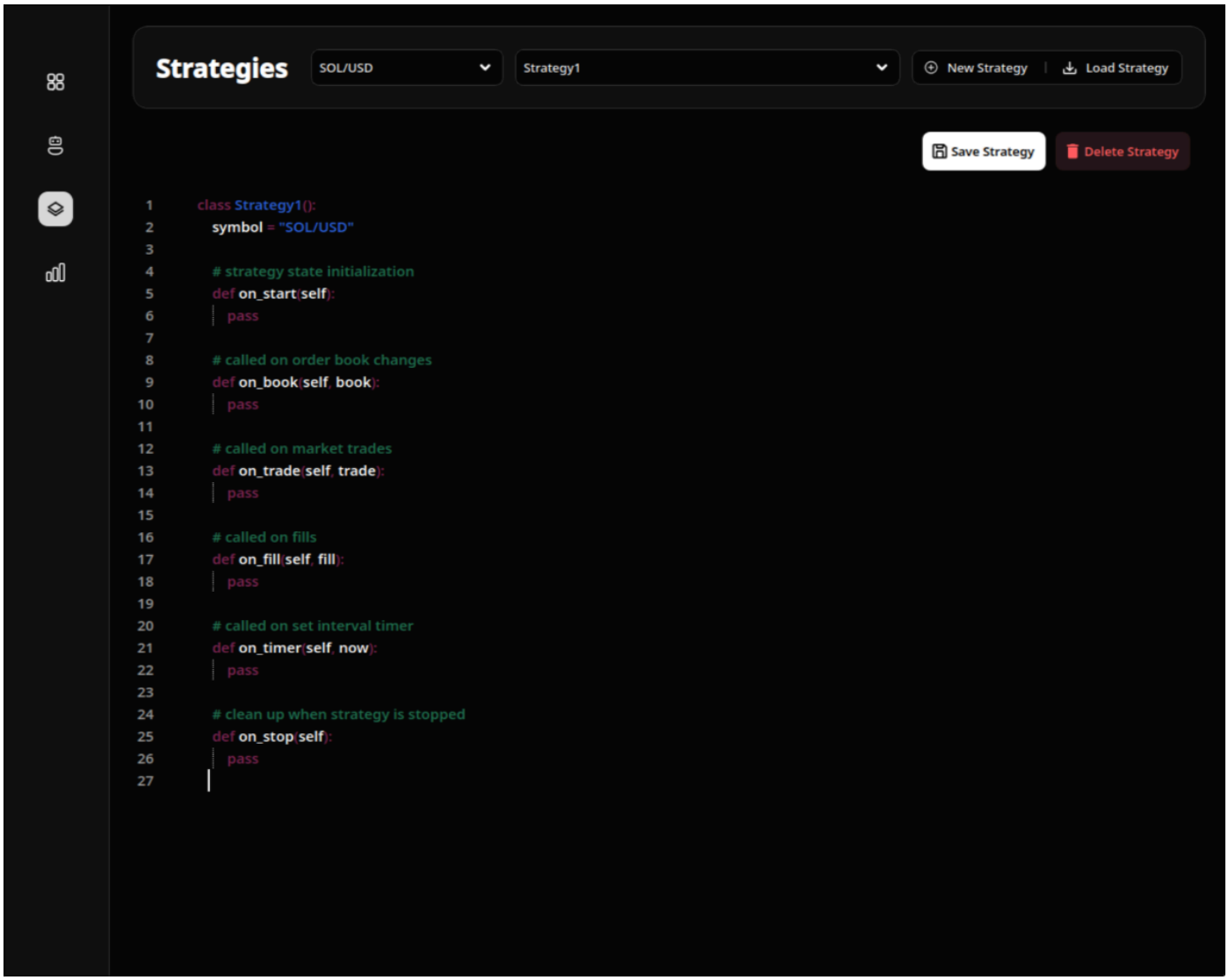
interact with strategies. The bot window allows “bots” to be created, which run automated trading strategies.



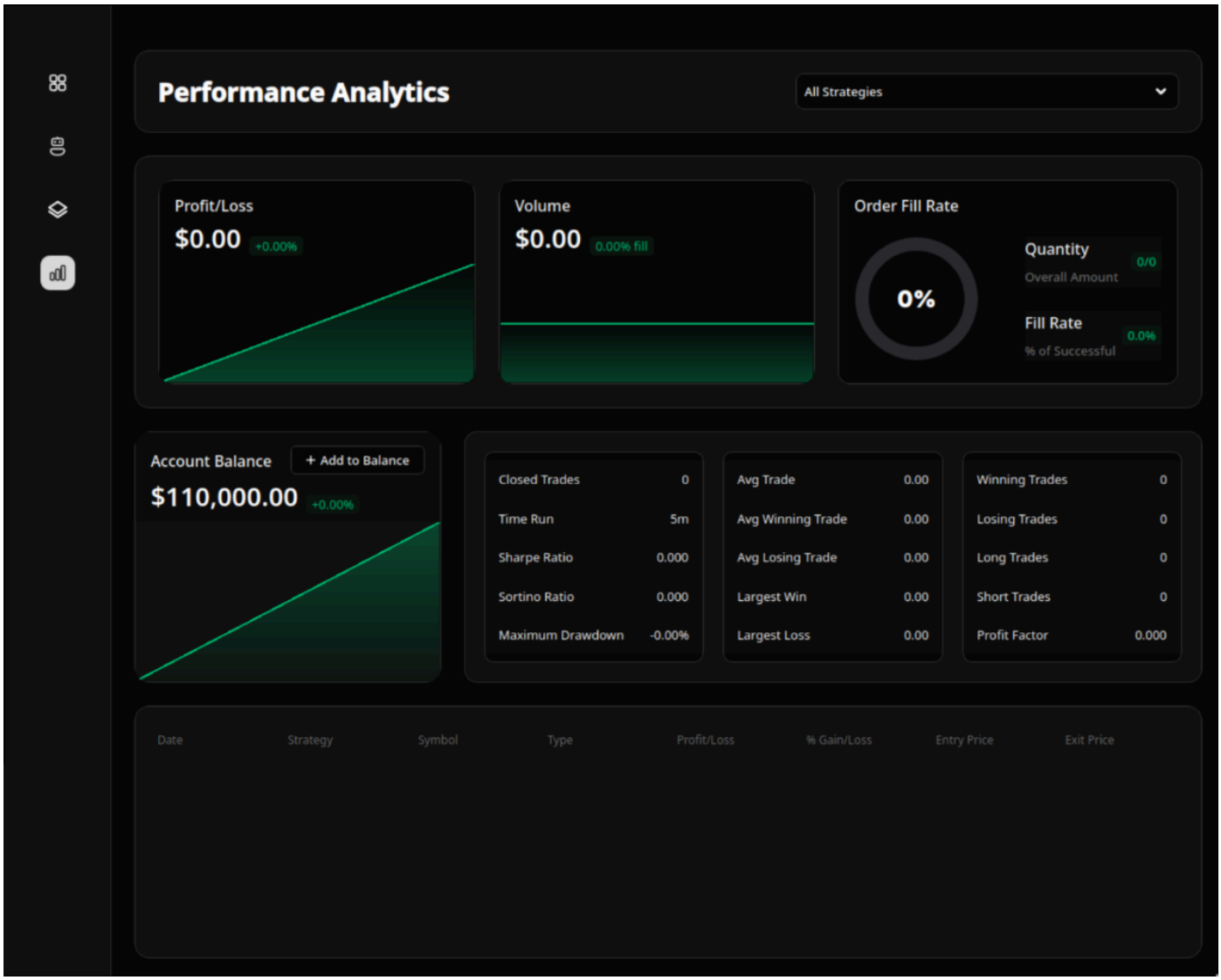
An example of a created bot running:



Bots run strategies, which are the algorithms used for them to make trades. They can be created in the next tab:



The client offers some methods that must be implemented for a bot.



The last page is the analytics page. It shows performance in the current session. This is how you can determine how your strategies are performing.

B. FAQ

Simulator

Q: What time model does the simulator use?

A: The simulator operates on a discrete-event model. Events are processed strictly in increasing timestamp order with a deterministic tie-breaking mechanism. Simulation time is used in place of wall-clock time, and it is only advanced upon processing an event.

Q: How are simultaneous events handled?

A: Simultaneous events are minimized by using nanosecond scale timestamping. However, in the event of a tie, tiebreaking is done according to an order ID, which is unique for every order. The simulator assigns all orders an ID immediately on receipt, before reordering according to timestamps. In the event of a tie, this ID is used as a tiebreaker.

Q: How is latency modeled?

A: Only inbound latency is modeled. Upon receiving a user order, its timestamp is assigned to the current sim time and incremented by a constant amount for latency. An additional increment is added for jitter, which can be configurable sampled using a uniform distribution or a normal distribution.

Q: Is the simulation deterministic?

A: Yes, given the same input stream of orders, the simulation will always execute exactly the same. Randomly generated data can be seeded to ensure a deterministic sequence of events.

Q: How is correctness verified?

A: Our build CI-CD requires that all unit tests pass to allow code to be committed to our release build. Currently, there is an extensive suite for the various components ensuring that they are correct.

Q: How does the simulator generate data?

A: Currently the simulator only supports two modes: randomly sampled and file replay. Randomly sampled data samples inter-arrival time from an exponential distribution, order types and sides from a uniform distribution, and price from a normal distribution. Randomly sampled data can also be seeded for reproducibility. Additionally, the simulator can record streams of events and write them to a binary-packed file, which can be replayed using file mode.

Q: How does the simulator differ from a real exchange?

A: There are many notable differences and simplifications from a real exchange. There is no hidden liquidity, no auctioning, no hidden trades, etc. Only simple order types like limits and markets are supported. Priority rules are simplified and only use strict price-time priority. Self-trading is not prevented.

Limit Order Book

Q: What book depth is maintained?

A: The engine maintains a full depth for all orders. The client uses a stripped down version of the limit order book for a local representation that tracks price level quantity and which level orders exist in.

Q: Which order types are supported?

A: The order book only supports limit orders and market orders, as well as cancels and updates.

Q: Which instrument types are supported?

A: Only large cap equities are supported.

Q: Does the limit order book support multiple symbols?

A: No, the order book is per-symbol. Currently, the simulation only supports a single symbol, however adding multisymbol is feasible, and would only require spawning multiple order books.

Matching Engine

Q: What matching algorithm is used to execute trades?

A: The limit order book uses a price-time priority, with strict FIFO priority within a price level. Ties are broken as stated above before reaching the order book; once an order has reached the book it relies on FIFO queue behavior. Per-order tracking is used for all orders in the book. Upon a cross, the crossing order is matched against the best price on the other side. This is repeated until the crossing order is completely filled or until the order no longer crosses.

Q: Are partial fills supported?

A: Yes, a trade can be partially filled and it will remain in the book with the remaining quantity.

Q: Are self trades allowed?

A: Currently, there is no self-match prevention logic.

Q: How are market orders handled?

A: Market orders sweep the book and execute trades until completely filled or until there is no liquidity.

Q: How are cancellations processed?

A: Cancellations are processed the same as all other orders, meaning that they are latency-delayed according to the configured latency.

User Interface

Q: What programming languages are supported for custom strategy execution?

A: At the moment, only strategies written in Python are supported.

Q: Where are strategy and bot information stored?

A: At initialization of the application, the system creates and stores a local SQLite database file that holds strategy, bot, and account balance information. The file is located in the gui/data folder of the project.

Q: How does the application handle multiple strategies?

A: Each strategy runs its own StrategyRunner, which maintains its state, position, and performance metrics independently.

Q: How does the application generate candlestick charts?

A: Trades are bucketed into time intervals, and each interval aggregates OHLVC (open, high, low, volume, close) values to form the candles.

FIX (Financial Information eXchange)

Q: What is the primary purpose of FIX in this system?

A: FIX is the protocol for point-to-point session management and order routing. It directs the bidirectional transmission of business and administrative messages over a persistent TCP connection between a client and the exchange's gateway.

Q: Why does FIX utilize TCP?

A: TCP is used to ensure reliable, ordered delivery of messages. The system relies on TCP's built-in acknowledgement and retransmission mechanisms to ensure strict sequence integrity and guarantee arrival.

Q: If TCP is reliable, why does FIX need session management?

A: TCP only ensures reliability for the duration of a single connection. If the connection drops, the TCP sequence resets. FIX Session Management provides persistence across multiple connections. It uses independent sequence numbers so that upon reconnection, a client can detect gaps in the sequence and use a ResendRequest to synchronize with the exchange.

Q: What FIX messages are fully supported by the system?

A: The system currently supports NewOrder, OrderCancel, ExecutionReport, OrderCancelReject, Logon, Resend Request, TestRequest, and Heartbeat messages.

MoldUDP64

Q: What is the primary purpose of MoldUDP64 in this system?

A: MoldUDP64 is the transport layer for market data dissemination. It encapsulates binary ITCH messages into UDP datagrams for multicast delivery over the network.

Q: How does MoldUDP64 provide reliability over UDP.

A: MoldUDP64 uses a 64-bit sequence number in the header of every packet. Clients monitor these numbers to identify any gaps in the sequence. When a gap is detected, the client sends a RequestPacket via a unicast UDP socket to the server's retransmission port to recover the missing data.

Q: How does the system send out large amounts of market data quickly?

A: MoldUDP64 uses packet batching, instead of sending one network message for every single market event, it groups multiple events into a single data packet.

ITCH

Q: What is the primary purpose of ITCH in this system?

A: ITCH is a binary data protocol used to communicate event-by-event market updates in the matching engine. It defines the message structures for Add Order, Order Executed, Order Cancel, Order Deleted, and other market events, allowing clients to track every change to the limit order book.

Q: What ITCH messages are fully supported by the system?

A: The system currently supports AddOrder, OrderCancel, OrderReplace, OrderExecutedWithPrice, and Resend messages.

VI. TRANSITION PLAN

A. Knowledge Transfer

The code-base contains docstrings for almost all the different components and methods. There are a few areas of note that might be useful for extension and maintenance:

Order: The key struct that every other component uses. These represent orders and contain information like the ID, timestamp, etc. There is an enum determining the type, which contains type-specific information.

EventSource: A trait implemented by all data sources. It has a single method that must be implemented (next_event() -> Option<Order>)

Simulator: A struct that owns the simulation, it manages simulation time, pacing, order ordering, and owns the limit order book. It can communicate to the two networking components via SPSC queues.

PyClient: A module that contains all the python bindings for the client. This module contains a lot of wrapping of other rust components, like Order. Anything that goes into the client from the Rust side must be defined in this module.

Despite being programmed in Rust, the design of the application largely adheres to an OOP style. Thus, the easiest way to get acquainted with the project is to read the documentation for the major components, which are: Simulator, Orderbook, FixEngine, FixHandler, MoldEngine, and EventSource.

B. Next Steps

There are a number of improvements that can be made, depending on the goals of those that continue the project.

One major improvement that could be made would be multi-symbol trading, allowing for modeling arbitrage. This could be done by allocating a limit order book per symbol. The FIX message for orders

already contains a currently unused identifier for symbols, so routing could be done to send orders to the right books.

The client could be improved to include queue position tracking. Currently the client tracks aggregates per price level, as well as hashmap that maps IDs to active orders. Notably, this does not provide any information about queue position, however it could be added by tracking liquidity in a price level before the user adds an order to that level and decrementing it as trades are executed at that price level.

Additional sources of simulated data can be implemented. The current model uses traits to allow for generic types to implement event generation. The simulation itself takes a function pointer to the next_event() function, so adding new event sources should be trivial. Currently, the simulation has poisson sampled random data and file based replay, but other useful sources include: historical data (L3 from an exchange), live data (translation of live data from an API), and agent-based (running strategies to generate L3 data).

Other instrument types could be implemented, though this would be more difficult than other changes due to the tight integration of the limit order book with the current order types. Additionally, additional message support would have to be added for the different types of orders associated with the new instrument.

VII. ACKNOWLEDGEMENTS

Special thanks to our advisor Ashish Aggarwal for sponsoring this project.

VIII. APPENDICES

XI. BIOGRAPHY

Kathryn Jerez is a fourth-year Computer Science student at the University of Florida, currently completing a Bachelor of Science. She has 2 years of professional experience in full-stack software development as a Software Engineer Intern, employed for 6 months at Purpose Financial and approximately a year and half at UKG. After graduation, her professional goal is to continue working in the field of full-stack software development as a Software Engineer I at UKG.

Jacob Nickerson is post-baccalaureate Computer Science student at the University of Florida, completing an additional major in Computer Science following a Bachelor of Science in Chemistry from the University of Florida. After graduation, he will continue working in systems software development as an associate software engineer at MRSL Real Time Systems Laboratory.

George Stewart is a fourth-year Computer Science student at the University of Florida, currently completing a Bachelor of Science following an Associates of Arts from Seminole State College of Florida. With a professional focus on Information Technology and Cybersecurity, he holds the CompTIA Network+ and Security+ industry certifications. His professional goal is centered on transitioning into Digital Forensics and Incident Response.